

# CS383 - Software Engineering

## Software Implementation (I)

# Outline

- Object-oriented design
- Design patterns with examples
- Software configuration management
- Open source development
- Code convention
- Code documentation

# Software design and implementation

- It is where the executable system is developed
  - Identify software components and relationships as detailed in design stage
  - Implement these components
- If the functionality of a subsystem exists elsewhere, you can obtain the off-the-shelf subsystem and use it.
  - Communication with and configuration of features of that system is studied to deliver current developing system

# Implementation paradigm

- Based on design goals
  - Distinguish the type of development paradigm(s) to follow
    - Object-oriented
    - Service-oriented
    - Event-driven
    - Model-driven
    - Test-driven
    - etc.

# Object-oriented design

- Develop a number of different system models
- Common creative activities of OO design process
  - Define the context and modes of use of the system
  - Design the system architecture
  - Identify the principal system objects
  - Specify object interfaces
  - Apply concepts like, inheritance, polymorphism, interfacing,

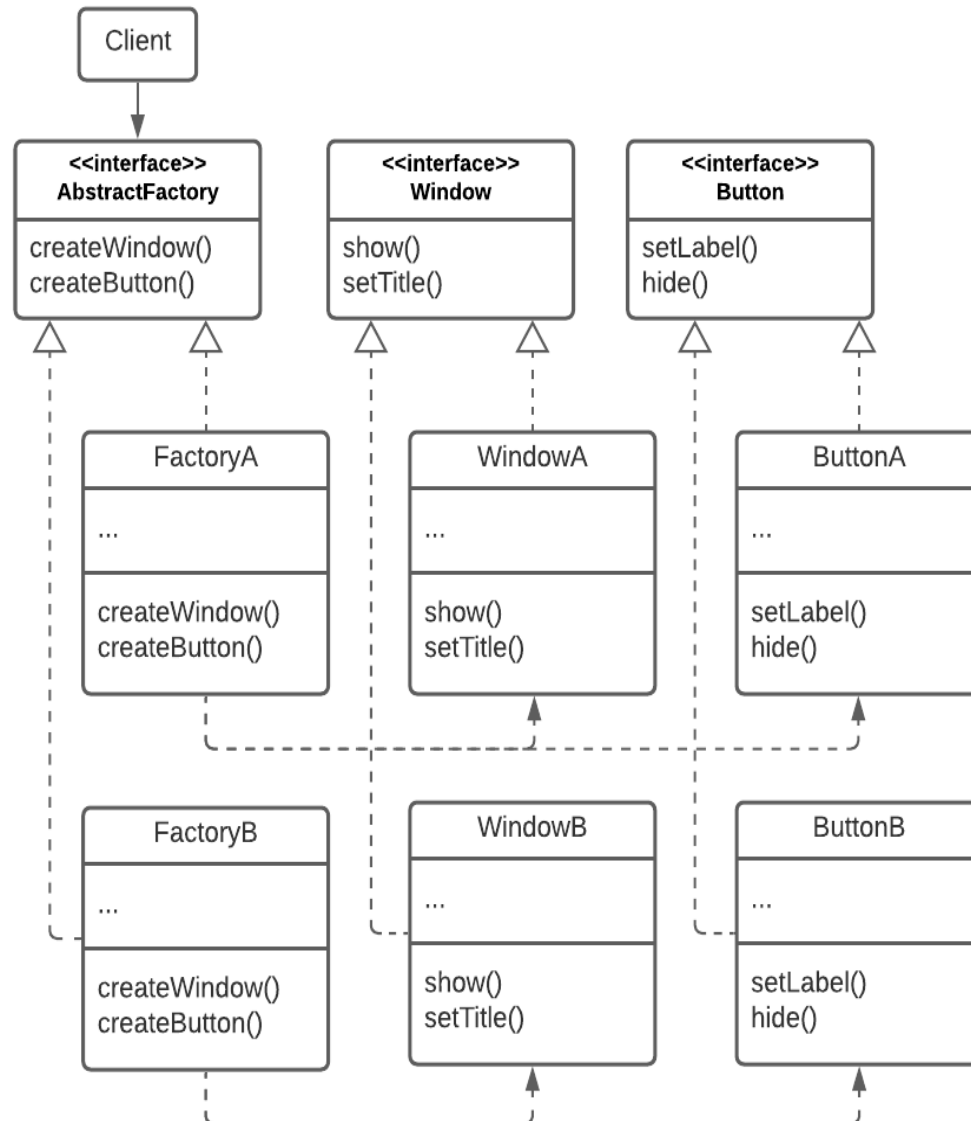
# Design patterns

- A collection of abstract knowledge of solutions of problems that can be reused.
- Design patterns can be:
  - Creational patterns
  - Structural patterns
  - Behavioral patterns

# Creational patterns: abstract factory

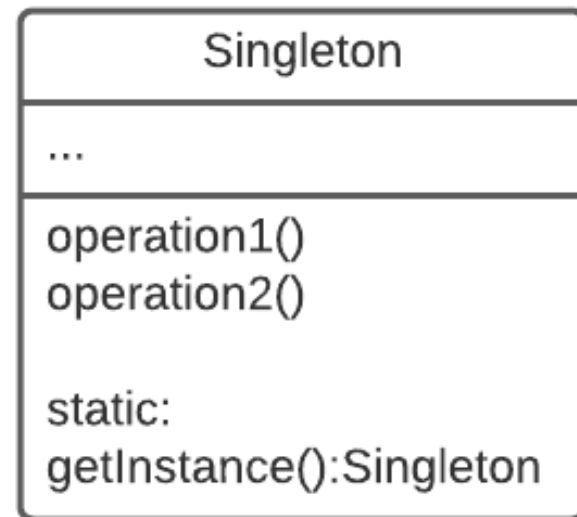
- Allows a program to create objects from related classes without knowing the classes are implemented
- Problems
  - A family of related classes can be used together
  - E.g, windows, frame, buttons can be used as parts of underlying window system provided by the operating system

# Creational patterns: abstract factory



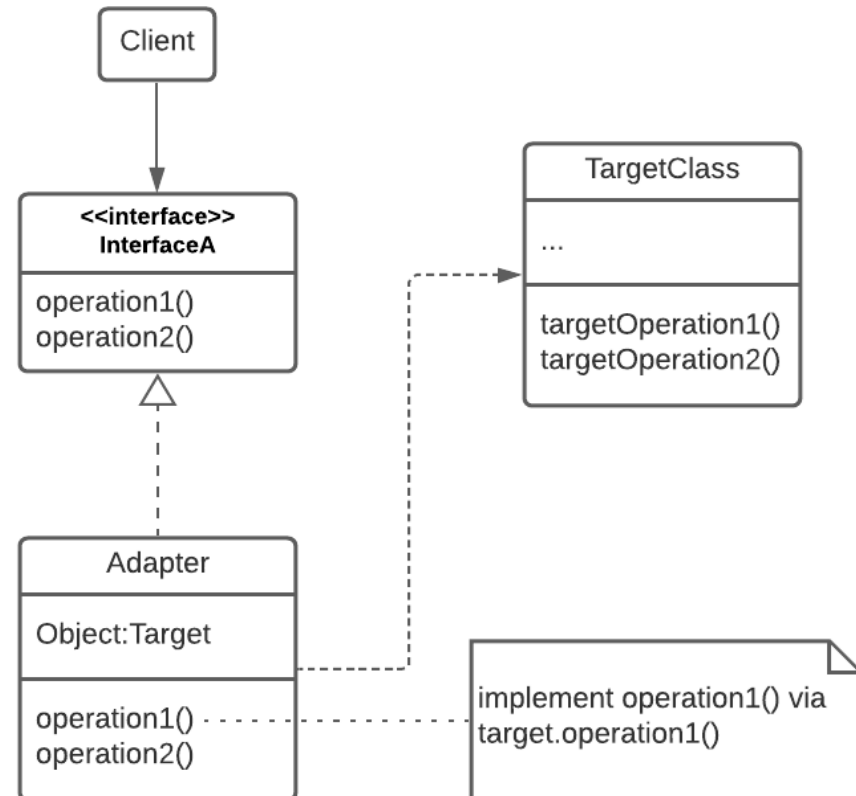
# Creational patterns: singleton

- Ensures that a class has only one instance/object.
- Problems:
  - Emergency calls 911
  - PO box of a large company



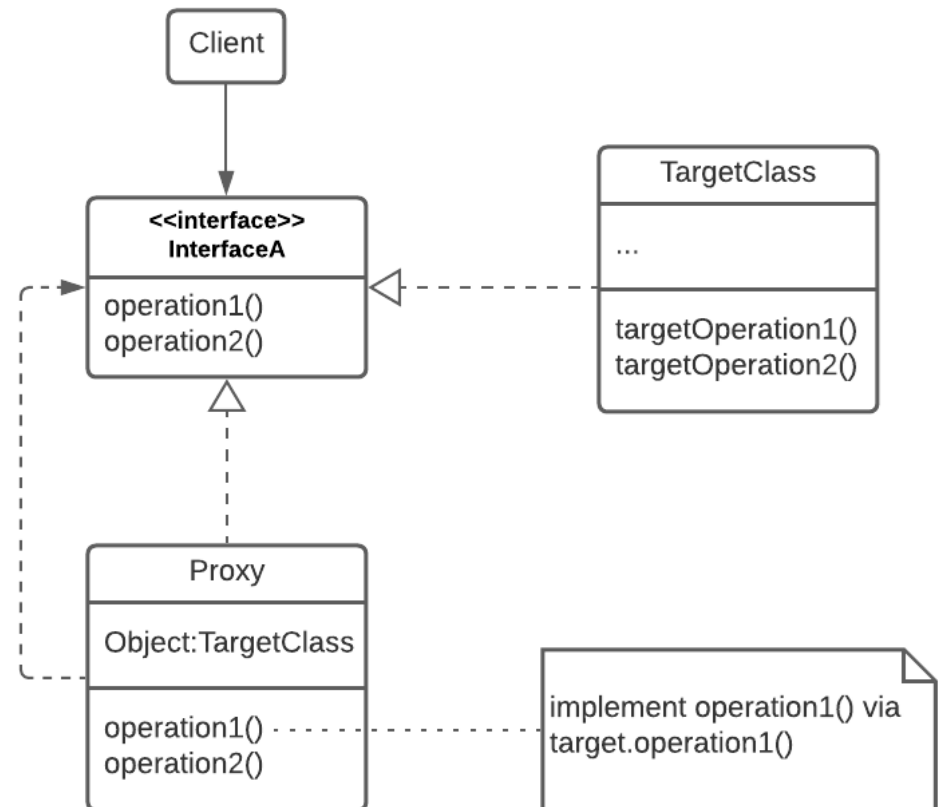
# Structural patterns: adapter

- Converts the interface of a class into another interface that the client expects. Also known as wrapper.



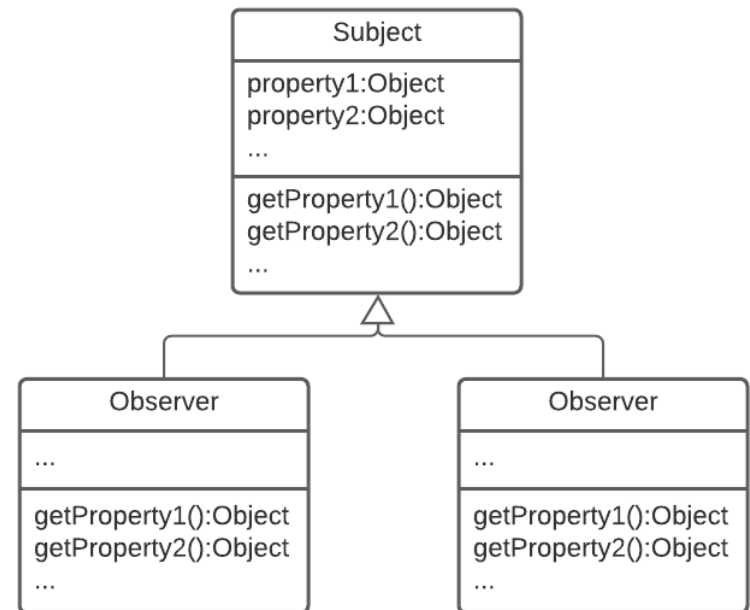
# Structural patterns: proxy

- As the name suggests, works as proxy for its client
- Eg. a proxy class delegates the client to connect to a remote DB



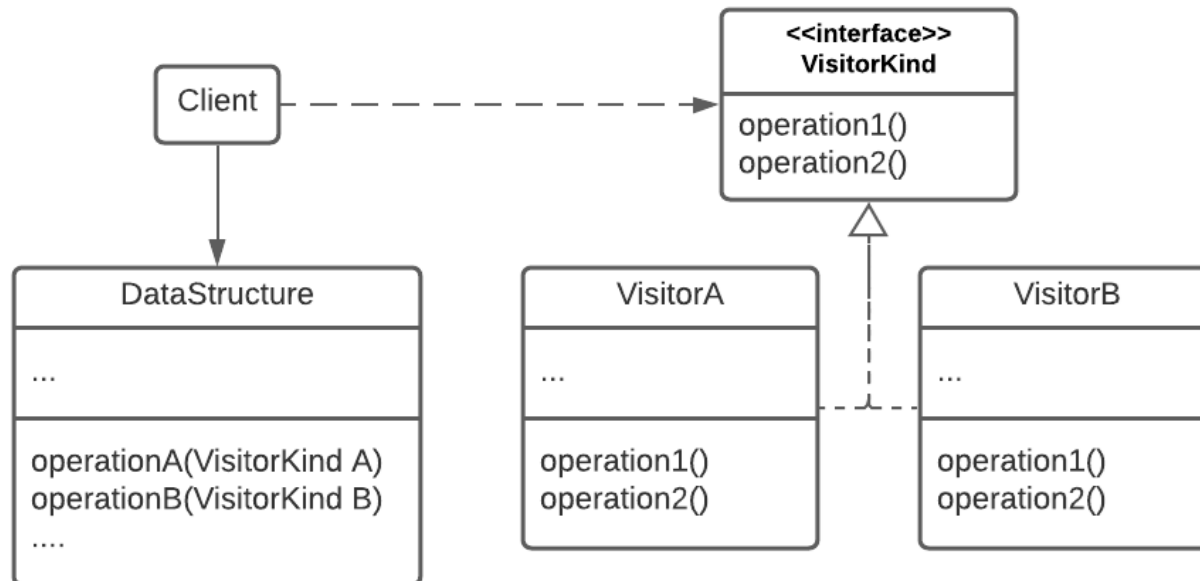
# Behavioral patterns: observer

- Allows presenting a data set from a particular view
- E.g.,
  - Share broker: different clients have specific interest with different stock
  - Student records: can have different presentation of the data: transcripts, registration, marketing



# Behavioral patterns: visitor

- Allows a processor to be passed to another processors as a parameter



# Configuration management

- Managing changing software using tools
  - Most popular ones: SVN and Git
- Support the following
  - Allow cooperation of code development
  - Support tools and system integration
  - Version control: using commits, changes of developing system can be managed
  - Support documentation of implementation development
  - Support batches, problems fix, and code comparison
  - Support issuing system releasing

# Open source development

- Source code of the software is published publicly and volunteers are invited to be part of the development
- Volunteers are usually interested or users of the produced software
- Volunteers can be recruit over the internet and can be in different locations.
- Most common software
  - Linux, Java, Ruby, Django

# Code convention

- Code conventions are important to programmers
  - 80% of the lifetime cost of a piece of software goes to maintenance
    - Obviously, the author of original code is not going to be maintaining the software for whole life of software
  - Why code convention is important?
    - code conventions improve the readability of the software
    - Allowing understandability of new code more quickly
    - Facilitate continuity

## Java Code convention (cont.): naming

- Interface classes: starts with 'I' followed by class name with the first letter capitalized. Eg. **ICar**, **IPublication**
- Implementation classes (realization of interfaces) should be suffixed with **Impl**

E.g. **FordCarImpl** (ie, implements ICar interface)

- Classes and abstract classes: starts always with capital letter
  - Book, Person, Socket

## Java Code convention (cont.): naming

- Methods: use the verb form of a word. If long name, use camelCase.

Eg. `getMaxNumber()`, `run()`, `initializeAllObjects()`

- Variables: use meaningful names but short too.
  - Ideally for code refactoring
  - Eg, `double average`, `String customerIPAddress`
- Constant variables: in block letters always separated with underscore
  - Eg. `public final int MINIMUM_ROWS = 5;`

## Java Code convention (cont.): blocks

- Opening of a block '{' should be in the same line of definition/declaration and the closing block should be in a new line after the last statement of the definition/declaration of the block

```
void printInfo(Student details){  
    //do stuff . . .  
    if(details.length>=1){  
        //do more stuff . . .  
    } else {  
        //do alternative more stuff . . .  
    }  
}
```

# Code documentation: Java

- A java class file should starts with copyright statement just before the package name

Eg.

```
/*  
 * %W% $E$ company name  
 *  
 * Copyright (c) 2016-2023 Sun Microsystems, Inc.  
 All Rights Reserved.  
 * This software is the confidential and  
 proprietary information of Sun  
 * Microsystems, Inc. ("Confidential Information").  
 You shall not  
 * disclose such Confidential Information and shall  
 use it only in  
 * accordance with the terms of the license  
 agreement you entered into
```

## Code documentation (cont.d): Java

- After that, a description of the class is given

Eg.

```
/**  
 * This class allows users to do this and that...  
 * @version      1.50 03 Oct 2017  
 * @author Firstname Lastname  
 */  
public interface ICar {  
    ...  
}
```

## Code documentation (cont.d): Java

- Method documentation: a method description must be provided right before its declaration. It gives details about the function and how to use it.
- All parameters of the function must be stated along with their types and each one must have a description along with its purpose.
- If the function returns, the return values and situations must be stated.
- If the function throws any Exceptions or Errors, all situations must be identified and explained how they are triggered.

Eg.

```
/**
 * This methods does/allows this and that... in
 * details
 *
 * @param stdId: student id must be provided,
 * otherwise the function returns
 * @param student mark, represent the mark of
 * student. If no mark available,
 * then 0 is assigned by default
```

## Code documentation (cont.d): Java

- Variables documentation
- Use single line comment (//) before or infront of variable declaration

Eg.

```
private Set<Student> stds = new  
HashSet<Student>() ; //store students objects  
  
// singleton variable to hold university single  
object  
public static QUUniversity INSTANCE;
```